

Multi-Agent Autonomous Dump Truck Simulator for Optimizing Paddock Dump Packing Density

Aakash Rajan

with Ritvik Ghosh and Aditya Gadepalli

Caterpillar Tech Challenge 2026 - Winners

Code: <https://github.com/Aakash-Rajan-27/dump-packing>

Jun 2026

Abstract

Autonomous haul trucks pack mine dump paddocks at roughly 7.4 m spacing between dumps, against about 3.0 m for staffed operations. The gap comes from the planning paradigm rather than the vehicles: a fixed grid of spot points and pre-planned lanes is laid down before any truck arrives, and must be over-spaced so that earlier piles are not mistaken for obstacles. We study the alternative in which no dump plan exists. Each dump location is chosen online from the current terrain, and each truck's route is planned and re-planned as the terrain changes. We formulate this as a closed loop of four coupled sub-problems: online placement under a physical terrain model, look-ahead to avoid self-blocking, fleet-to-target assignment, and kinematically feasible multi-agent motion planning. The system combines an angle-of-repose sandpile terrain model, a phase-adaptive six-term placement score, Monte Carlo Tree Search over dump sequences, Hungarian assignment with a turning-radius-aware cost, hybrid A* for pose-constrained reversing, and Conflict-Based Search over oriented-rectangle footprints. A repulsive, ant-colony-style pheromone field gives placement a decaying memory of recent dumps and traffic. It runs as a two-thread Python simulator over arbitrary polygons with mixed fleets. Single-agent operation is stable end to end; multi-agent coordination is implemented and functional but not yet robust under heavy contention. We describe the method, the integration and concurrency design, the planners we weighed, a log of the liveness fixes that worked and failed, an evaluation protocol that is being instrumented, and a planned neural Large Neighbourhood Search extension.

1 Introduction and problem formulation

The central claim of this work is that dump-packing density is limited by planning before execution, not by truck capability. Today's two-step method fixes spot points over the paddock polygon, then fixes lanes to each one. The problem statement [?] identifies three failure modes of that method:

1. **Mixed fleets break fixed spacing.** A grid sized for one truck class leaves low spots when a smaller truck dumps to it, and arrival order for mixed payloads cannot be planned in advance.
2. **Irregular polygons leave dead zones.** A regular lattice never tiles an irregular boundary, so corners and edges are left unfilled.
3. **Defensive over-spacing.** Spot points are separated widely enough that earlier piles are not detected as obstacles. This is the main source of the 7.38 m versus 3.03 m spacing gap reported in the brief.

1.1 Setting

A paddock is a simple polygon P with one entry point e on its boundary. Trucks arrive in an order not known in advance. Truck i has class $k(i)$ with payload m , footprint $w \times l$ and minimum turning radius R . Each dump deposits volume $V = m/\rho$, with bulk density $\rho \approx 1.8 \text{ t/m}^3$ for waste rock. The terrain state is a height field $z(x, y)$ over P .

At each decision epoch, with a set $\mathcal{I}$ of idle trucks, the system must choose dump targets, match trucks to targets, and produce collision-free trajectories that each truck can drive. It must satisfy the brief’s hard constraints:

C1 Containment: no part of a truck leaves P while dumping.

C2 Connectivity: no pile may disconnect any unfilled region of P from e .

C3 Kinematics: paths respect each class’s turning radius, and trucks reverse into the dump face.

C4 Safety and liveness: concurrent trucks never collide and never deadlock.

C5 Mixed-fleet robustness: density holds for arbitrary sequences of payloads.

The objective is packing quality, measured as mean nearest-neighbour spacing between dump centres and as volumetric fill against a target height. Throughput is a secondary objective.

1.2 Why this is hard

The problem couples four sub-problems that are usually studied apart. Online placement is a sequential decision with delayed consequences, since a good dump now can seal off a pocket later. Assignment is combinatorial across the fleet. Motion planning is non-holonomic and pose-constrained, because trucks must arrive backwards at a heading. Multi-agent planning must also run on a map that changes after every dump, which invalidates any precomputed roadmap.

1.3 Contributions

- A closed-loop architecture for plan-free dump packing that treats every placement as an online decision over a physically grounded terrain model.
- A placement policy that combines a phase-adaptive heuristic prior with MCTS look-ahead over dump sequences, penalising futures that isolate unfillable pockets.
- A connectivity check made cheap by testing reachability on a coarsened grid, with a local pre-filter that skips most checks.
- A repulsive pheromone field, adapted from ant colony optimisation, that spreads consecutive dumps in time and steers placement away from congested haul routes.
- A planning stack that pairs Hungarian assignment, hybrid A* to a sampled staging pose with a straight reverse, and CBS with separating-axis footprint conflicts.
- A snapshot-based two-thread architecture, and an honest record of the safety and liveness interventions tried in development, including those that failed.
- An open simulator for arbitrary polygons and mixed fleets, with an evaluation protocol under construction.

2 Related work

This work sits where online packing, search-based decision making and multi-agent path finding (MAPF) meet. Each component builds on established methods; the contribution is their coupling on a terrain that the agents themselves reshape.

Spot-point packing. Industrial practice fills a polygon with staggered or radial lattices of spot points. These are static packings, sized for one truck class, and cannot react to the realised pile shapes or to mixed payloads.

Granular terrain. Piles are modelled as cones at the angle of repose, followed by slope-limited relaxation. This is a deterministic relative of the Abelian sandpile model [?]. It is coarse compared with discrete-element simulation, but cheap enough to call inside a search loop.

Search with heuristic priors. We use UCT, Monte Carlo Tree Search with the UCB1 bandit rule [2, 6]. A heuristic score restricts the branching to a few promising children, the same pattern AlphaGo used with a learned policy prior [?].

Ant colony optimisation and pheromone fields. Ant colony optimisation (ACO) coordinates many simple agents through *stigmergy*: agents modify a shared medium, the pheromone field, and later agents read it instead of communicating directly [4?]. Each trail is reinforced by good solutions and decays through evaporation, which keeps the search from locking onto stale decisions. Ant Colony System added local trail updates that make recently used components less attractive to other ants, which spreads the colony across the search space [?], and MAX-MIN Ant System bounded trail values to avoid premature convergence [?]. Rajendran and Ziegler applied ACO to permutation flowshop scheduling, with trail intensities over job-position pairs, evaporation, reinforcement from the best sequence found, and local search over the constructed sequences, to minimise makespan and total flowtime [10]. That work shows that a decaying shared memory can guide sequential, combinatorial placement decisions, which is how we use it. Our field differs in one respect: it is *repulsive*. Fresh dumps and passing trucks lower it, and it recovers over time, so it plays the role of ACS’s local update, steering later trucks away from areas that were just used or are congested (Section 4.3).

Assignment. Matching idle trucks to targets is the linear assignment problem, solved exactly by the Hungarian method [7, 9].

Non-holonomic planning. Hybrid A* searches over continuous vehicle states reached by kinematically valid arcs [3], using the bicycle model for curvature limits.

MAPF. Conflict-Based Search splits planning into a constraint tree and single-agent replanning [12]. Extensions handle agents with volume [8] and kinematic limits [5]. Our CBS uses oriented-rectangle footprints with a separating-axis test [?]. For scale, Large Neighbourhood Search repeatedly re-solves subsets of agents [?], and Neural LNS learns which subsets to choose [14]; this is our planned extension (Section 9).

Gap. We know of no published system that closes the loop between terrain-modifying placement decisions and MAPF on the same changing map, with mixed vehicle geometry and a connectivity constraint on the free space.

3 World model: terrain physics and dual-resolution grid

Every downstream decision reads a height field that behaves like dumped rock, so the simulator cannot gain density by ignoring real pile geometry. The polygon is rasterised into 1 m cells. Each cell carries a height, a state (empty, partial, filled, reserved, path-reserved, protected, boundary) and a pheromone value.

3.1 Pile geometry

A dump of volume V forms a cone whose slope equals the angle of repose θ , set to 35° for waste rock. Volume and slope fix the base radius:

$$V = \frac{1}{3}\pi r^2 h, \quad h = r \tan \theta \quad \implies \quad r = \left(\frac{3V}{\pi \tan \theta} \right)^{1/3}. \quad (1)$$

Height $(r - d) \tan \theta$ is added at each cell a distance $d < r$ from the centre. A local relaxation pass then caps the height difference between 4-neighbours at $\Delta \tan \theta$, where Δ is the cell size, moving half the excess downhill, for up to 25 sweeps. Overlapping piles therefore merge and spread as granular material would, rather than stacking as columns.

Pile height is derived, not tuned: $V = m/\rho$ with $\rho = 1.8 \text{ t/m}^3$. An earlier version used a per-class height-per-dump constant; replacing it with the derivation removed a free parameter and made mixed-fleet behaviour follow from payload alone.

3.2 Two resolutions

A fine grid (1 m) supports cone geometry, low-spot detection and smooth paths. A coarse grid of 3 m blocks supports flood fills and connectivity checks. Fill percentage is the share of valid cells at the 5 m target height. A second measure, pack percentage, is the mean clipped height over the target height across all valid cells.

3.3 Drivability

For each truck class, a heading-indexed drivability tensor of size $\text{rows} \times \text{cols} \times 8$ marks whether the truck’s full rectangle fits at each cell and heading, clear of the boundary and of piles above a clearance threshold. Entry-corridor cells are protected from dumping but remain drivable; excluding them from the mask made every route from the gate infeasible in an early version.

4 Dump-point selection

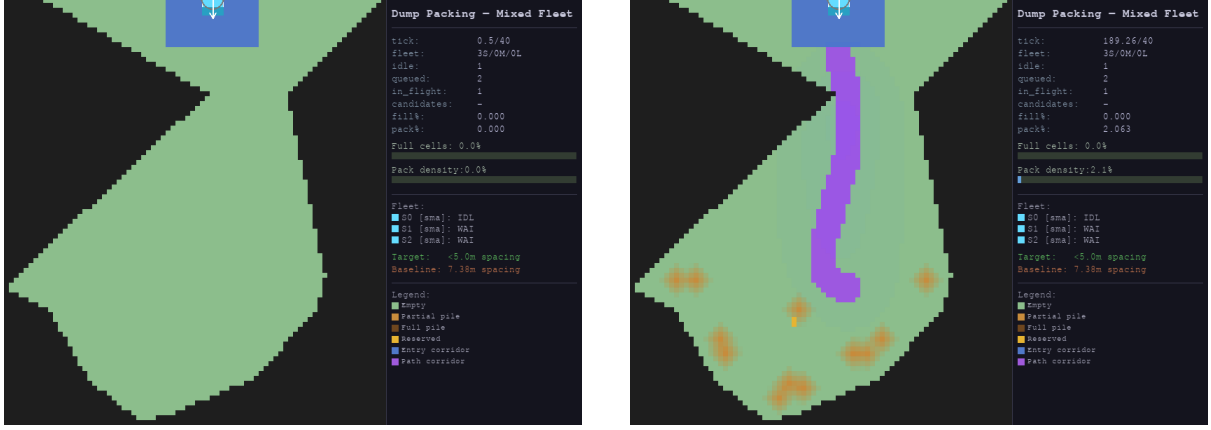
Placement is a three-stage funnel: feasibility filters prune the polygon to legal targets, a cheap heuristic ranks them, and MCTS re-ranks the top 50 by simulating future dumps.

4.1 Candidates and feasibility

The grid is divided into stride blocks, and the lowest dumpable cell in each block becomes a candidate. This biases candidates toward depressions, which directly targets the low-spot failure mode of fixed grids.

Two checks enforce the hard constraints. **Containment (C1)** requires the truck’s full rectangle, positioned with its rear at the dump cell, to fit inside P at some reversing heading. Using rear-anchored rather than centre-anchored geometry was necessary; centre-anchored checks wrongly rejected valid cells along the boundary.

Connectivity (C2) asks whether every free region stays reachable from the entry after a hypothetical dump. A full-grid BFS per candidate is too expensive. Two observations make it cheap:



(a) A randomly generated paddock at tick 0.

(b) Overlapping piles after several dumps.

Figure 1: World model in the simulator. (a) Fresh irregular polygons on each run test the planner against shapes that lattices tile badly. (b) Slope relaxation merges neighbouring cones into a continuous surface.

1. Disconnection is topological, so it can be tested on the 3 m coarse grid, provided the block size is below the smallest gap the test must resolve. This divides the BFS cost by roughly the block area, nine times here.
2. A cell can only disconnect the space if it is *pinched*, meaning at least two of its four neighbours are blocked. Unpinched cells skip the BFS entirely, which is the common case.

When fill exceeds 70%, the order flips: every candidate is BFS-checked before scoring, since chokepoints become frequent.

4.2 Heuristic score

Each candidate cell c receives a weighted sum of six normalised terms:

$$\begin{aligned}
 s(c) = & w_1 \underbrace{e^{-d_{\text{pile}}/2\Delta}}_{\text{density}} + w_2 \underbrace{(1 - \bar{f}_{24})}_{\text{coverage}} + w_3 \underbrace{\frac{\max(0, z^* - z)}{z^*}}_{\text{height gap}} \\
 & + w_4 \underbrace{\phi}_{\text{pheromone}} + w_5 \underbrace{\frac{d_{\text{centre}}}{d_{\text{max}}}}_{\text{boundary}} + w_6 \underbrace{(1 - e^{-d_{\text{entry}}/50})}_{\text{entry distance}}
 \end{aligned} \tag{2}$$

Here d_{pile} is the distance to the nearest material, $\bar{f}_{24}$ is the material fraction in a 24 m window, z^* is the target height, d_{centre} is the distance from the grid centre and ϕ is the pheromone field defined in Section 4.3.

4.3 The pheromone term

The pheromone term gives the placement policy a short-term memory of where the fleet has just been. The other five terms read only the terrain, so without it consecutive decisions taken a few ticks apart see almost the same score map and send trucks to the same neighbourhood, where they queue behind each other and contend for the same approach lanes.

The field $\phi(r, c) \in [0, 1]$ starts at 1 everywhere, meaning undisturbed. Two events lower it:

- **Dumping.** Every cell whose height rises in a dump's footprint is set to $\phi = 0$.

- **Traffic.** At each step a truck at cell (r_0, c_0) lowers the field by a Gaussian trail,

$$\phi(r, c) \leftarrow \max\left(0, \phi(r, c) - s \exp\left(-\frac{(r-r_0)^2 + (c-c_0)^2}{2\rho_t^2}\right)\right), \quad (3)$$

with strength $s = 0.5$ and radius $\rho_t = 3$ m.

Once per tick the field recovers and diffuses. Each cell closes a fixed share of its deficit, and the result is smoothed with a Gaussian kernel G_σ of $\sigma = 3$ m:

$$\phi \leftarrow \text{clip}_{[0,1]}(G_\sigma * [1 - \lambda(1 - \phi)]), \quad \lambda = 0.85. \quad (4)$$

The deficit $1 - \phi$ therefore decays geometrically, by 15% per tick, while diffusion spreads it to neighbouring cells so that the signal covers an area rather than a single cell.

Because it enters Equation (2) with a positive weight, a high ϕ attracts: the score favours cells that have been quiet for a while. This is the inverse of the classic ACO trail, in which reinforcement attracts later ants to a good solution [4, 10]. Here a fresh dump or a passing truck is a reason to go elsewhere, as in the local update of Ant Colony System [?]. The field has three effects:

1. **Temporal spreading.** Consecutive dumps are pushed apart in time, so a new pile is not placed beside one that is still being formed and approached.
2. **Congestion avoidance.** Trail deposits mark active haul routes, so placements that would send another truck through busy lanes score lower.
3. **Bounded memory.** Geometric recovery forgets old activity after roughly ten to fifteen ticks, so areas become attractive again once traffic has moved on, much as evaporation keeps ACO from converging prematurely [?].

The pheromone is weighted 0 in the early phase, when coverage and entry distance dominate, and 0.1 in the mid and late phases (Table 1). It is a tie-breaker that separates otherwise similar candidates, not a primary objective. Because it is read from the same snapshot as the terrain, it also costs nothing extra to pass to the planner thread.

4.4 Phase-adaptive weights

No fixed weighting suits a whole run, so the weights switch with fill percentage (Table 1). This is a simple, interpretable phase-based policy. The thresholds and weights were hand-tuned on the reference polygon and are a natural target for learning (Section 9).

Table 1: Phase-adaptive weights $(w_1, \dots, w_6)$ for Equation (2).

Phase	Trigger	Dens.	Cov.	Height	Pher.	Bound.	Entry	Intent
Early	fill < 3%	0	100	0	0	10	100	Push material deep and wide before the gate silts up
Mid	3–10%	0.3	0.2	0.2	0.1	0.1	0.1	Cluster and fill gaps
Late	fill ≥ 10%	0.5	0	0.3	0.1	0.1	0	Density and surface evenness

4.5 MCTS look-ahead over dump sequences

The heuristic is myopic: the top-scored cell can be the last reachable cell beside a wall it is about to seal. MCTS corrects for this by evaluating *sequences* of dumps rather than single choices.

The root holds the current terrain, and its children are the top 50 candidates. A node at depth k is a hypothetical k -th dump. Selection uses UCB1 with $c = \sqrt{2}$:

$$\text{UCB1}(n) = \frac{W_n}{N_n} + c \sqrt{\frac{\ln N_{\text{parent}}}{N_n}}. \quad (5)$$

Expansion is progressive and heuristic-guided: a node expands only after its first visit, adding the 6 best next moves at depth 1 and the 3 best at depth 2. Rollouts then apply up to 20 random dumps from the remaining fillable cells, each raising height by the requesting class’s per-dump increment. The terminal state is scored as

$$J = \text{packing} - 0.05 \frac{|\{\text{unfilled cells with } \geq 3 \text{ filled neighbours}\}|}{|\{\text{unfilled cells}\}|}, \quad (6)$$

which rewards density and penalises pockets that will be hard to fill later.

For N idle trucks of one class, the system walks down the most-visited path for N levels and returns that sequence of targets. Visit count is used rather than mean value because, under UCB1, visits are the search’s own measure of confidence. A batch decision uses 200 simulations; a single truck at the gate uses 100. Classes are planned largest first, since large trucks have the fewest feasible targets.

5 Fleet assignment and kinematic motion planning

Assignment is solved exactly, and each truck then plans a path that it can physically drive, ending in a reverse into the dump face.

5.1 Hungarian assignment

Greedy nearest-target matching lets two trucks claim the same cell and produces poor global plans. Matching idle trucks to selected targets is the linear assignment problem, solved exactly in $O(N^3)$ with `scipy.optimize.linear_sum_assignment`. This is real-time for fleets up to about 15 trucks. The cost is

$$C_{ij} = w_d \frac{\|p_i - q_j\|}{D_{\max}} + w_h \frac{R_i}{R_{\max}} \frac{|\psi_i - \beta_{ij}|}{\pi}, \quad (7)$$

where ψ_i is the truck heading, β_{ij} is the bearing to the target, R_i is the class turning radius and $w_d = 1.0$, $w_h = 0.4$. Scaling the heading term by $R_i/R_{\max}$ makes a wide-turning truck pay more for a poor approach angle, since it has less room to correct. Euclidean distance is a proxy; true path cost comes from the planner below.

5.2 Fleet model

Table 2 lists the truck classes used. All values are configuration parameters; any mix and any new class can be added without code changes.

Table 2: Mixed-fleet truck classes. Dump volume is $V = m/\rho$ with $\rho = 1.8 \text{ t/m}^3$.

Class	Payload (t)	Width $\times$ length (m)	Min. turning radius (m)	Dump volume (m^3)
Small	45	5.5×8.7	3	25
Medium	133	7.1×11.6	9	74
Large	227	9.8×14.8	33	126

5.3 From grid path to drivable trajectory

The coarse planner is A* over (row, col, heading) with 8 heading buckets and an explicit turning penalty, so its paths are already close to feasible. A smoother then converts the cell path to a rear-axle trajectory under the bicycle model, where wheelbase L and steering angle δ fix the turning radius

$$R = \frac{L}{\tan \delta}. \quad (8)$$

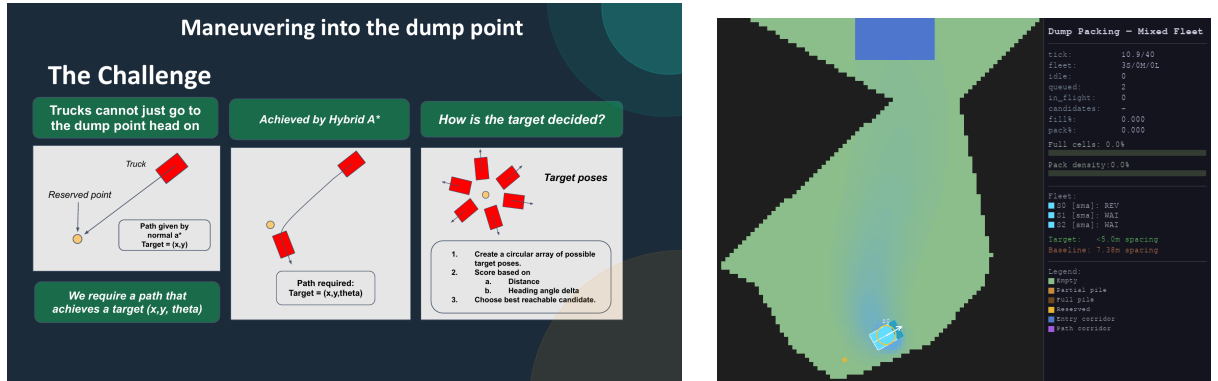
The smoother looks one radius ahead and clamps any target that would require a tighter turn than the class minimum.

5.4 Pose-constrained arrival: staging and reversing

Haul trucks back up to the dump face, so the goal is a pose (x, y, θ) , not a point (Figure 2). We decompose arrival into a forward leg and a straight reverse:

1. **Staging-pose sampling.** 24 candidate poses (every 15°) are placed on a ring around the target at the pile-clearance radius plus 3 m. Each pose faces away from the target. Poses that are not drivable or lack a clear straight reverse line are discarded. The rest are scored by distance (weight 0.1) and heading change (weight 1.0) from the truck’s current pose.
2. **Hybrid A* to the staging pose.** Nodes are continuous (x, y, θ) states reached by bicycle-model arcs, so the output is drivable by construction and hits the final heading exactly.
3. **Straight reverse** in 0.5 m steps until the rear is at the target, within 0.5 m and 10° tolerance.

This mirrors operator practice and reuses the generate–score–select pattern of dump-point selection at the scale of a single approach.



(a) Point goal (plain A*) versus pose goal (hybrid A*). (b) A truck reversing from its staging pose.

Figure 2: Pose-constrained arrival. Reversing to a dump face requires reaching a heading, not just a position.

6 Multi-agent coordination

Independently optimal paths routinely intersect in space and time, so truck paths are planned jointly with Conflict-Based Search, using exact rectangle geometry for conflicts.

6.1 Conflict-Based Search

Low level. Space-time A* over (row, col, heading, t), with a wait action, finds the cheapest path for one truck that avoids a set of forbidden (cell, t) pairs. Waiting costs 4 times a move, so a detour of up to 4 cells is preferred to standing still. The horizon is 150 steps.

High level. A constraint tree is searched best-first by total cost. The root plans each truck alone. At the first conflict between trucks a and b , two children are created: one forbids a from the conflict state, the other forbids b , and only the constrained truck is replanned. The first conflict-free node is returned.

Two conflict types are checked. A *vertex conflict* is body overlap at the same time step. An *edge conflict* is two trucks swapping positions between consecutive steps. Because trucks are rectangles, overlap uses the separating-axis theorem on the two oriented footprints, each inflated by a 1 m buffer (a Minkowski sum). Grid proximity is necessary but not sufficient for a conflict, so the SAT test prevents trucks in adjacent lanes from being falsely flagged.

CBS is complete and optimal when the tree is searched exhaustively. For real-time use we cap the search at 200 nodes and fall back to the best node found, which gives up the guarantee under heavy contention. Trucks returning to the gate get time-offset path bookings, so the space-time reservations stay temporally correct.

Section 7.3 compares CBS with the other multi-agent planners we weighed.

6.2 Run-time safety and liveness

Planning runs against frozen snapshots, so live state can drift before a plan is applied. Three run-time layers protect against this:

1. **Proactive conflict detection.** Each tick, committed paths are checked ahead for the first future conflict. A detected pair is replanned in place by CBS, with every other truck's path locked as a hard constraint.
2. **Physical guard.** After every motion step, an SAT test on all truck pairs rolls back any move that would overlap another body.
3. **Liveness recovery.** Two detectors trigger operator-style manoeuvres, described next. A freeze detector releases any truck that still makes no progress, frees its reservations and returns it to the assignment pipeline.

Deadlock: two trucks jammed. A truck is *stuck* when the physical guard has rolled back its moves for four consecutive ticks. When two stuck trucks are closer than half the sum of their lengths, one of them reverses along an arc of about 90° until its body clears the other truck's width, and falls back to a straight reverse if the arc is blocked. The truck with the longer remaining path is chosen to give way, because a detour costs it the least in relative terms; if its escape is infeasible the roles are swapped. The reversing truck releases its dump reservation and corridor and re-enters the pipeline once the escape ends.

Headlock: two trucks nose to nose. A truck is *stagnant* when its net displacement stays under 0.5 m for 20 ticks. Two stagnant trucks within $0.75\times$ the sum of their lengths whose headings differ by more than 120° ($\cos \Delta\psi < -0.5$) are treated as a head-on meeting. The truck with the longer remaining path performs a yield: a short reverse followed by a 90° turn, prepended to its current path so that it resumes its route afterwards.

Cooldowns. Both manoeuvres set a cooldown on both trucks, about the length of the manoeuvre, during which they are excluded from conflict scans and from re-detection, so a new manoeuvre cannot fire before the current one has finished.

These are engineering mitigations that keep the system live. They are not a verified deadlock-free protocol, and the escape manoeuvres are not themselves checked by CBS against other trucks.

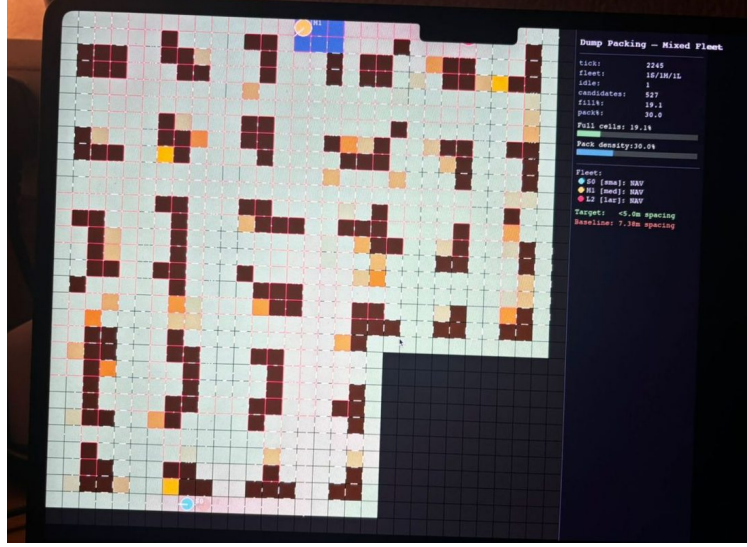


Figure 3: A mixed fleet (small, medium, large) routing at the same time, each truck planned by CBS against the others' reserved corridors.

7 System design, integration and engineering lessons

Most of the engineering effort went into making the algorithms of Sections 4 to 6 work together on a map that changes under them. This section records the architecture, the integration choices, the planners we weighed, the concurrency design, and the liveness fixes we tried, including those that failed.

7.1 Architecture

The system is about 5,500 lines of Python (NumPy, SciPy, Shapely, Pygame) across 20 modules, split into two threads so that heavy planning never stalls motion or rendering (Figure 4).

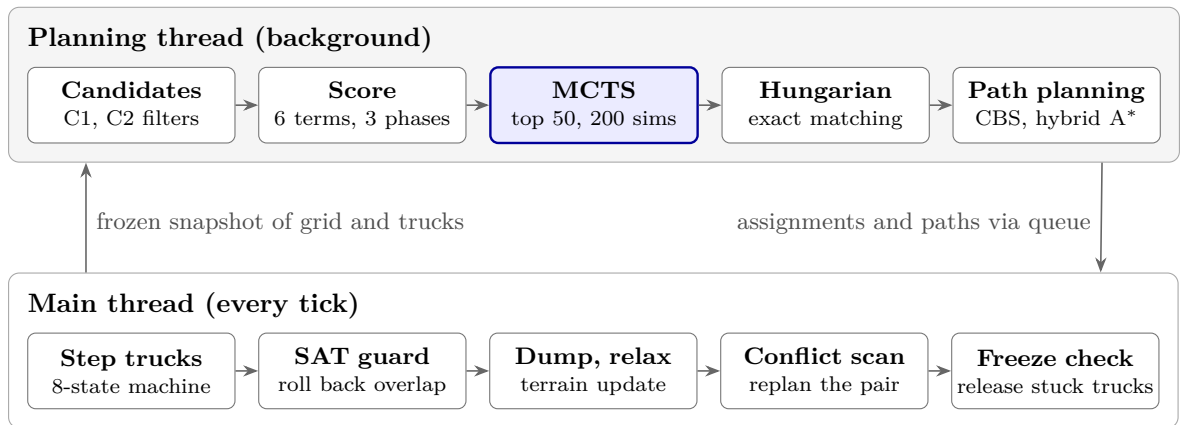


Figure 4: System architecture. Planning runs on frozen snapshots in a background thread, so the motion and rendering loop never waits for it.

The planning thread takes a frozen snapshot of the grid and fleet, runs the placement and planning stack, and posts assignments and paths to a queue. The main thread advances every truck through an eight-state machine (waiting, entering, idle, navigating, reversing, dumping, exiting, leaving), applies the physical guard, deposits material and runs conflict and freeze checks.

Every tunable sits in one configuration file: polygon, entry point, fleet mix, truck classes, cell size, material parameters, scoring weights, MCTS budget, CBS node cap and wait cost. A procedural generator produces random irregular polygons from a seed, so experiments run on reproducible but varied sites.

7.2 Integrating the algorithms

A narrowing funnel with typed hand-offs. Each stage passes a small, well-defined object to the next and cuts the problem size by roughly an order of magnitude, so the expensive stages only ever see a few inputs. Several thousand grid cells become one candidate per stride block, then a heuristic ranking, then the top 50 for MCTS, then N targets for N idle trucks, then N truck–target pairs and finally N paths. The heuristic is cheap and vectorised, MCTS is the most expensive stage per candidate, and the planners run once per truck.

Heuristic as a prior, search as a correction. The six-term score is not used to choose directly. It orders the children MCTS expands (6 at depth 1, 3 at depth 2), so search effort concentrates where the heuristic is already confident and the rollouts correct for its myopia. To keep rollouts cheap, they raise cell heights by a fixed per-dump increment instead of running the full cone deposition and relaxation of Section 3; the real terrain model is applied only to dumps that actually happen.

Where versus who. MCTS decides *where* to dump from terrain alone, and the Hungarian algorithm decides *which truck* goes where. Folding truck identity into the tree would multiply its branching by the fleet size. Classes are planned largest first, because large trucks have the fewest feasible targets and smaller trucks can fill the remaining gaps. Targets already claimed by a class are removed before the next class is planned.

Distance proxy in the assignment cost. The Hungarian cost uses Euclidean distance and heading rather than true path length. Exact costs would require N^2 planner calls per decision, and the planners themselves already handle the actual route.

Shared state as the contract between ticks. Stages communicate across ticks through cell states rather than direct calls. A chosen target becomes *reserved* so that no other decision can claim it. A planned route marks a *path-reserved* corridor, which other trucks may still dump on in later epochs but which the renderer and conflict checks can see. The entry corridor is *protected* from dumping but stays drivable.

One low-level planner, four callers. The same space-time A* and CBS code serves batch planning for idle trucks, pre-planning for trucks waiting at the gate, in-place replanning of a conflicting pair with all other paths locked, and exit planning. Fixes and constraints therefore apply everywhere at once.

Derived rather than tuned parameters. Wherever possible, parameters follow from physics or geometry: pile size from payload, density and angle of repose; staging radius from pile radius; heading penalty from turning radius; and window sizes expressed in metres so that the grid resolution can change without retuning.

7.3 Alternative path planners

We weighed the planner families in Table 3 against four requirements of this problem: rectangular agents with turning limits, a single narrow entry shared by all traffic, a map that changes after every dump, and fleets of 3 to 15 trucks that must be planned in well under a second.

Table 3: Path-planning approaches weighed for the single-truck and multi-truck problems.

Approach	Strength	Decision and reason
<i>Multi-truck coordination</i>		
Independent A* per truck	Fast and simple	Rejected: no coordination, so paths collide
Prioritised planning [? ?]	Cheap; each truck plans around the reserved paths of higher-priority trucks	Rejected as the main planner: incomplete, and a fixed priority order fails in the single-gate throat where an inbound and an outbound truck must pass
Joint space-time A*	Complete and optimal	Rejected: the joint state space grows exponentially with fleet size
Conflict-Based Search [12]	Optimal; pays for joint reasoning only where trucks interact	Chosen , with a node cap for real-time use
Bounded-suboptimal CBS (ECBS) [1]	Much faster under contention, bounded cost loss	Future work; a natural fit for the dense late phase
Reactive avoidance (ORCA) [13]	Decentralised and smooth	Rejected: no guarantee against deadlock in corridors, and weak for non-holonomic rectangles
<i>Single-truck motion</i>		
Grid A* with heading and smoothing	Fast; turning penalty keeps paths near-feasible	Chosen for haul legs
Hybrid A* [3]	Drivable by construction; reaches an exact pose	Chosen for the staging pose before reversing
Dubins / Reeds–Shepp curves [11]	Exact shortest paths with reversing in free space	Not used alone: ignores piles and other trucks; a candidate analytic expansion inside hybrid A*
Sampling (RRT*) [?]	Handles continuous space	Rejected: no native time dimension for multi-agent use, slow convergence in narrow gaps between piles, and randomness hurts reproducible A/B runs

Two lessons shaped the final planners. First, the search state must stay minimal: an early low-level A* that carried the parent cell in its state, (r, c, r_p, c_p) , expanded to around 68 million nodes. Heading buckets carry the same turning information with far fewer states. Second, time must be explicit wherever trucks share space. Trucks returning to the gate are booked with time offsets, so that the space-time reservations CBS checks against are temporally correct.

7.4 Concurrency and parallelisation

From synchronous to background planning. In the first version, planning for all idle trucks ran inside the simulation tick, and every truck froze while MCTS and CBS ran. We moved planning to a background thread that works only on *snapshots*: a shallow copy of the grid with deep-copied arrays for state, height, pheromone and corridors, and frozen copies of each truck. A work queue and a result queue are the only channels between threads. The main

thread is the sole writer of live state, so no locks are needed. An in-flight set prevents a truck from being planned twice, and a result is discarded if its truck is no longer idle when it arrives.

The cost of snapshots. A plan is computed on the world as it was and applied to the world as it is. Two plans that are each conflict-free can clash once applied, and a pile dumped after planning can land on a route. The proactive conflict scan and the physical guard (Section 6) exist largely to absorb this drift.

The GIL. Python’s global interpreter lock lets only one thread run Python bytecode at a time, and both threads are CPU-bound. The planner thread therefore competes with the simulation loop rather than running in parallel with it, and can lag several ticks behind. For deterministic testing we added a synchronous mode that plans inline in a headless, seeded run, so logic changes can be compared run against run without thread timing in the result.

Vectorisation. The largest speed-ups came from NumPy rather than threads. Building the dumpable mask and the MCTS base set had been a Python double loop over about 8,100 cells; it is now a single vectorised expression. Cone deposition is vectorised over the pile’s bounding box, and scoring uses distance transforms and uniform filters. Slope relaxation is still a Python loop and dominates run time; a 1,200-tick headless run takes about 250 s.

Planned: real parallelism. The next step is a process pool, which sidesteps the GIL and can use all cores of a 32-thread workstation. Its prerequisites are that snapshots pickle cleanly and that every module re-imports safely under the spawn start method used on Windows; a thread pool is the fallback. Process-level parallelism also fits root-parallel MCTS, where independent trees are searched from the same root and their visit counts summed, and the LNS subproblems of Section 9, which are independent by construction.

7.5 What we tried for safety and liveness

Most of the remaining difficulty is not finding a path but keeping a dense fleet both safe and moving. Table 4 logs the main interventions. The numbers in it come from an experimental branch with a stricter per-frame validator (oriented-rectangle collisions, boundary breaches, driving over piles, curvature and freezes), seed 42 and 1,200 ticks, and are development measurements rather than benchmark results.

Three lessons follow. First, the fleet’s traffic is a fragile equilibrium: almost every local safety fix we tried traded away liveness, so every change has to be judged on throughput, freezes and violations together, never on one alone. Second, the remaining over-pile violations have identifiable causes rather than random ones. The reverse-dock goal accepts any pose whose rear is within reach of the target, including poses where the rear has overshot, so a truck can straddle its own dump; and exit routes go stale when other trucks dump on them after planning. The planned fix tightens the goal to a near-side shell with the nose pointing away from the pile, and replans exit routes at the supervisor level rather than per frame. Third, a safety layer that silently absorbs failures also hides them, which is why we are adding the intrusion metrics and bypass mode of Section 8.

8 Current status and evaluation protocol

The single-agent pipeline is stable end to end; the multi-agent layer works but is not yet robust under heavy contention, and quantitative benchmarks are still being instrumented. We therefore report qualitative behaviour and the protocol we are building, not headline numbers.

Table 4: Safety and liveness interventions and their outcomes during development.

Intervention	What happened	Outcome
Collision guard on shared path-corridor cells	Too conservative: paths that share cells at different times are safe, but trucks were stopped anyway	Replaced by SAT on oriented footprints
Spatial stale-check that cancelled gate pre-plans	False positives on cells shared at different times caused cancel-replan loops, so trucks never entered	Removed; rely on CBS's time-indexed plans
SAT guard applied inside the entry corridor	An idle truck at the gate permanently blocked an exiting truck	Corridor exempted in the guard; logged separately for metrics
Excluding protected entry cells from the drivable mask	Every route from the gate became infeasible	Protected cells kept drivable
Gate release by distance and path-length test	Race: an exiting truck met an inbound one nose to nose in the throat; throughput collapsed after about 3 dumps	Directional mutex: hold inbound trucks while any truck is in transit. 12 dumps, 0 freezes
Hard dump-spacing keep-out	Dumps $12 \rightarrow 15$, but freezes $0 \rightarrow 2$; over-pile events unchanged	Reverted
Live guard that reverts any step climbing onto a pile	Livelock: dumps $12 \rightarrow 2$, trucks trapped against the pile they had just made	Reverted
Greedy truncation of the reverse-dock search	Worse final poses than the cost-ordered search; freezes $0 \rightarrow 2$	Reverted
Exit planner allowed to drive over current piles	Exit routes crossed fresh mounds	Piled cells made non-drivable; over-pile $26 \rightarrow 22$, freezes still 0

8.1 Maturity

Table 5: Maturity of each layer.

Layer	Components	Status
Single agent	Terrain model, candidates and filters, scoring, MCTS, Hungarian assignment, bicycle-model smoothing, hybrid A* staging and reversing	Stable: a lone truck, or trucks that never contend for space, reliably completes dump sequences
Multi agent	CBS, proactive conflict detection, physical guard, escape manoeuvres, freeze recovery	Functional; robustness under dense contention is the active work

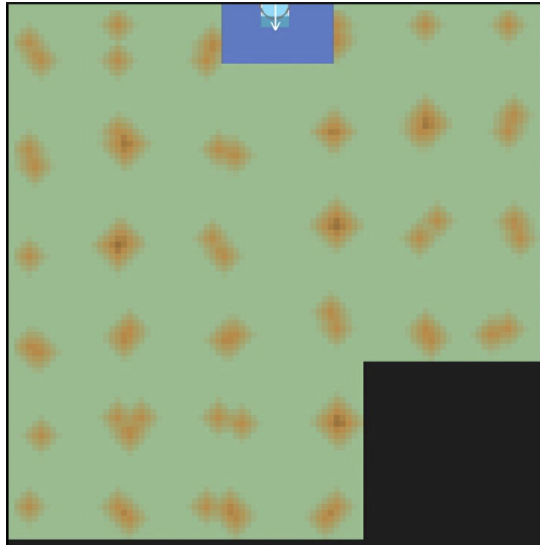


Figure 5: Early-phase dump distribution on the L-shaped reference paddock. The coverage and entry-distance terms spread the first piles across the far reaches of the site, including the corners a lattice would miss.

Figure 5 shows the qualitative effect of the early-phase weights.

8.2 Metrics

The simulator computes fill percentage, pack percentage and a running mean nearest-neighbour spacing between dump centres, shown live against the brief’s 3.03m staffed and 7.38m autonomous references. A logging layer is being added to persist the metrics in Table 6 per run.

Intrusions are not measurable today, because the physical guard rolls back every overlap before it can be counted. A benchmark-only bypass mode lets overlaps occur and logs them instead, exposing the planner’s true conflict rate. Runs are seeded so that identical seeds give identical logs, which every A/B comparison requires.

8.3 Planned experiments

1. **Placement ablation.** Greedy top-score versus MCTS, and fixed versus phase-adaptive weights, measured by final spacing, pack percentage and the number of unreachable pockets.
2. **Baseline comparison.** Staggered and radial spot-point lattices on the same polygons and fleets.

Table 6: Metrics persisted per run.

Metric	What it measures
Nearest-neighbour spacing (mean, median, std, p90)	Packing quality, including the worst case
Fill and pack percentage over time; ticks to target fill	Density convergence and throughput
Driven $\div$ straight-line distance	Path efficiency
Truck-truck intrusions (momentary and sustained)	Coordination failure rate
Replans, freeze recoveries, escape manoeuvres	Coordination effort
CBS nodes, latency, fallback rate at the node cap	Scalability; fallback is a leading indicator of intrusions

3. **Mixed-fleet robustness (C5).** Random arrival sequences of payload classes; we expect spacing variance to stay low if placement adapts to realised piles.
4. **Scalability.** Intrusions, fallback rate and latency as the fleet grows from 3 to 15 trucks, across random polygons from the generator.
5. **Hyper-parameter sensitivity.** MCTS budget and depth, CBS node cap and wait cost.

9 Limitations and future work

The largest open problem is multi-agent robustness, and our main planned extension is a learned outer loop that decides which trucks to replan and when.

9.1 Limitations

- **Bounded CBS loses its guarantees.** With a 200-node cap, dense contention can return a best-effort plan that still conflicts.
- **Stale snapshots.** Plans computed on a snapshot can clash once applied to live state, even when each was conflict-free.
- **Unverified escapes.** Retreat and yield manoeuvres are hand-written and not checked against other trucks; they are the most likely source of real body overlaps.
- **Hand-tuned policy.** Phase thresholds, weights and the 0.05 pocket penalty were tuned on one reference polygon.
- **Myopic rollouts.** Rollouts ignore travel time and congestion, so MCTS optimises the final terrain rather than the joint placement-and-traffic cost.
- **Simplified physics.** Cone deposition with slope relaxation ignores compaction, tipping dynamics and material variability. The sim-to-real gap has not been measured.
- **Perception assumed.** The system reads a perfect height map; the brief allows for pile-aware perception, which we have not modelled.

9.2 Neural Large Neighbourhood Search

Conflict resolution today is reactive and pairwise: it fires only when a specific pair is about to collide. We plan to wrap CBS in an LNS outer loop following MAPF-LNS [?] and Neural LNS [14]. Periodically, subsets of trucks are proposed (by random choice, by spatial proximity, or by local congestion read from the pheromone field), a learned model scores them, and only the top few are re-solved by CBS with all other trucks locked.

We are explicit about what this can achieve. Fewer and smaller CBS calls reduce computation directly. Collisions fall only indirectly, because CBS is already conflict-free within any subproblem. We expect gains through three mechanisms: earlier, fleet-wide detection; a mandatory re-check of each result against live state before it is applied; and less congestion, so fewer escape manoeuvres fire. This gives a falsifiable prediction: if intrusions fall but escape-manoevre counts do not, the causal story is wrong.

Given single-digit fleets and CPU-only hardware, we start with a linear model on handcrafted subset features, trained on 50 to 100 seeded headless runs, and move to a deep model only if the linear one plateaus.

9.3 Further directions

- **Learned placement priors.** Replace the hand-tuned score with a policy trained from MCTS visit counts, in an AlphaZero-style loop, so search improves its own prior.
- **Decentralised operation.** The problem allows peer-to-peer collaboration. Distributed MCTS or market-based assignment could remove the central planner.
- **Formal liveness.** Prioritised or reservation-based protocols with provable deadlock freedom for the single-entry topology.
- **Perception in the loop.** Height maps from noisy sensor point clouds with semantic pile segmentation.
- **Middleware-agnostic interfaces.** Clean planning interfaces so the stack can sit behind a proprietary fleet-management system rather than a specific robotics middleware.

References

- [1] Max Barer, Guni Sharon, Roni Stern, and Ariel Felner. Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem. In *Proceedings of the Symposium on Combinatorial Search (SoCS)*, 2014.
- [2] Cameron B. Browne, Edward Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez, Spyridon Samothrakis, and Simon Colton. A survey of Monte Carlo tree search methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1):1–43, 2012.
- [3] Dmitri Dolgov, Sebastian Thrun, Michael Montemerlo, and James Diebel. Path planning for autonomous vehicles in unknown semi-structured environments. *The International Journal of Robotics Research*, 29(5):485–501, 2010.
- [4] Marco Dorigo, Vittorio Maniezzo, and Alberto Coloni. Ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1):29–41, 1996.
- [5] Wolfgang Hönig, T. K. Satish Kumar, Liron Cohen, Hang Ma, Hong Xu, Nora Ayanian, and Sven Koenig. Multi-agent path finding with kinematic constraints. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.
- [6] Levente Kocsis and Csaba Szepesvári. Bandit based Monte-Carlo planning. In *Proceedings of the European Conference on Machine Learning (ECML)*, pages 282–293, 2006.
- [7] Harold W. Kuhn. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.

- [8] Jiaoyang Li, Pavel Surynek, Ariel Felner, Hang Ma, T. K. Satish Kumar, and Sven Koenig. Multi-agent path finding for large agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [9] James Munkres. Algorithms for the assignment and transportation problems. *Journal of the Society for Industrial and Applied Mathematics*, 5(1):32–38, 1957.
- [10] Chandrasekharan Rajendran and Hans Ziegler. Ant-colony algorithms for permutation flowshop scheduling to minimize makespan/total flowtime of jobs. *European Journal of Operational Research*, 155(2):426–438, 2004. doi: 10.1016/S0377-2217(02)00908-6.
- [11] J. A. Reeds and L. A. Shepp. Optimal paths for a car that goes both forwards and backwards. *Pacific Journal of Mathematics*, 145(2):367–393, 1990.
- [12] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.
- [13] Jur van den Berg, Stephen J. Guy, Ming Lin, and Dinesh Manocha. Reciprocal n-body collision avoidance. In *Robotics Research*, volume 70 of *Springer Tracts in Advanced Robotics*, pages 3–19. Springer, 2011.
- [14] Zhongxia Yan and Cathy Wu. Neural neighborhood search for multi-agent path finding. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.